



**UNIVERSITÉ
FRANÇAISE
D'ÉGYPTÉ**

Faculty of Engineering

Department: Computer Engineering

Project title name: Skin Cancer Detection

Project Market name: Derma.AI

Project Sector: Health Care (SDG3)

Supervised by: Dr.Ahmed Magdy

Presented by: Amira AwadAllah

Skin Cancer Detection

Abstract:

Skin cancer remains one of the most prevalent forms of cancer globally, with early detection playing a crucial role in patient outcomes. This project presents a dual-platform AI-powered solution for skin cancer detection, consisting of a clinic-oriented website and a mobile application for public use. Both platforms utilize a convolutional neural network (CNN) model trained to classify skin lesions into three categories: melanoma, nevus, and basal cell carcinoma. The website is tailored for healthcare professionals and provides detailed diagnostic outputs, including class-wise confidence percentages and an interpretability heatmap to assist clinicians in understanding the AI's decision-making process. The mobile application, on the other hand, offers a simplified user interface aimed at helping individuals perform preliminary checks of suspicious skin lesions. The AI model achieves an accuracy of over 97%, making it a promising tool to support early detection and improve clinical workflow. This project bridges accessibility and medical reliability by offering AI assistance to both professionals and non-experts in a user-friendly manner.

Introduction:

Skin cancer is a growing public health concern, accounting for a significant portion of diagnosed cancers each year. The effectiveness of treatment is heavily dependent on early detection, yet access to timely dermatological evaluation remains limited in many regions. In response to this challenge, artificial intelligence has emerged as a powerful tool for image-based medical diagnostics, particularly in the field of dermatology.

This project introduces a comprehensive skin cancer detection system with two distinct but interconnected interfaces: a web platform for medical practitioners and a mobile application for the general public. Both interfaces leverage the same deep learning model, which was trained on a curated dataset of dermoscopic images representing three common skin cancer types: melanoma, nevus, and basal cell carcinoma. The model provides classification results with high accuracy and interpretable outputs.

The web interface caters to clinical environments by offering diagnostic transparency. It features a heatmap that highlights the lesion regions most influential in the AI's decision, and presents ranked predictions with confidence levels. This supports doctors in making informed decisions and fosters trust in the AI system. In contrast, the mobile application is designed for non-specialists, enabling users to capture or upload images and receive simplified diagnostic feedback. This empowers users to monitor their skin health and seek medical attention when needed.

Problem Statement:

Skin cancer is a major global health concern, with rising cases and survival rates closely tied to early detection, as illustrated by comparative diagnostic data. Conventional methods, such as visual assessments and biopsies, are often slow, expensive, and heavily reliant on clinician expertise, resulting in diagnostic delays and inconsistencies. These challenges are more severe in under-resourced areas, where specialist access is limited, increasing the likelihood of late-stage detection and poor prognosis. The global rise in melanoma, nevus, and BCC cases underscores the urgent need for automated diagnostic solutions to support early detection, enhance precision, and reduce mortality.

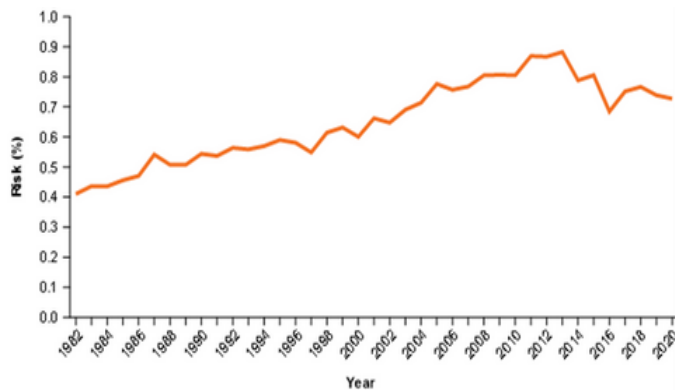


Figure1 Graph showing skin cancer prevalence worldwide.

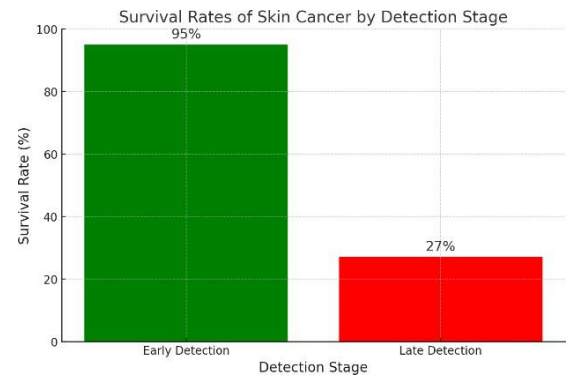


Figure 2 A contrasting chart of survival rates with early vs. late detection.

Prior solutions for skin cancer detection

Over the years, various approaches have been developed to improve the diagnosis of skin cancer. These methods range from traditional clinical techniques to advanced technologies employing artificial intelligence. Below is an overview of these prior solutions:

1. Traditional Methods

- **Visual Inspection by Dermatologists:**
Dermatologists have traditionally relied on visual examination of skin lesions using tools like dermatoscopes. While this method is non-invasive and accessible, it is heavily dependent on the expertise and experience of the clinician, leading to variability in diagnostic accuracy.
- **Biopsies:**
Considered the gold standard for definitive diagnosis, biopsies involve removing a sample of tissue for histopathological analysis. However, this approach is invasive, time-consuming, costly, and may cause patient discomfort.

2. Rule-Based Systems

- Techniques like the **ABCD rule** (Asymmetry, Border irregularity, Color, Diameter) and **7-point checklists** have been used to standardize the visual evaluation of skin lesions. These methods aim to reduce reliance on clinician intuition but still involve subjective judgment and have limited scalability.

3. Handcrafted Features with Machine Learning

- Early computational approaches used handcrafted features (e.g., texture, color, and shape) extracted from lesion images.
- Machine learning algorithms, such as support vector machines (SVM) and decision trees, were employed to classify skin lesions based on these features.
- While promising, these methods were constrained by the quality of feature extraction and were unable to match human-level performance consistently.

4. Deep Learning-Based Solutions

- Recent advances in deep learning have revolutionized skin cancer detection. Convolutional Neural Networks (CNNs) have shown remarkable ability to automatically learn complex patterns from large datasets of skin lesion images.
- Pre-trained models like **VGGNet**, **ResNet**, and **MobileNet** have been adapted for skin cancer classification tasks, achieving performance comparable to dermatologists in some studies.
 - **VGGNet**: High accuracy but computationally expensive and unsuitable for resource-limited settings.
 - **ResNet**: Excellent performance due to deeper architecture but requires significant computational resources.
 - **MobileNet**: Lightweight and efficient, making it ideal for mobile and embedded applications, albeit with a slight trade-off in accuracy.

5. Mobile Applications and Telemedicine Solutions

- Mobile apps incorporating AI models, such as **SkinVision** and **Miiskin**, allow users to capture images of skin lesions and receive preliminary risk assessments.
- These solutions enhance accessibility but may lack the accuracy required for definitive diagnoses.

6. Hybrid Models

- Combining clinical data with image-based AI models has been explored to improve diagnostic accuracy. By integrating patient history, lesion location, and demographic data, hybrid systems aim to provide more comprehensive assessments.

Limitations of Prior Solutions

While these solutions have advanced skin cancer detection, several challenges remain:

- High dependency on large, labeled datasets for training AI models.
- Variability in image quality and conditions during data acquisition.
- Limited deployment of advanced AI models in low-resource settings due to computational demands.
- Need for real-time and non-invasive solutions to ensure patient comfort and timely diagnosis.

Need for Improvement

The limitations of these prior solutions emphasize the need for:

- **Lightweight, scalable models** that can operate on mobile and embedded devices.
- **Non-invasive, real-time tools** that provide instant results, reducing reliance on specialists and laboratory infrastructure.
- **Accessibility-focused designs** to ensure usability in underserved regions with limited resources.
- **Efficient data augmentation and synthetic data techniques** to address dataset imbalances and improve model robustness across diverse populations.

Innovative solutions that balance accuracy, efficiency, and accessibility are essential to overcoming these challenges and improving early detection, patient outcomes, and global healthcare equity.

Solution

To address the challenges in skin cancer detection, I developed a mobile application that utilizes a **pretrained MobileNetV2 model** to predict three types of skin cancer: **melanoma, nevus, and basal cell carcinoma (BCC)**. MobileNetV2 was selected for its lightweight architecture, efficiency, and ability to deliver high performance with relatively low computational requirements, making it ideal for mobile applications.

1.Objectives of the solution

1. To develop a robust machine learning model capable of detecting three types of skin cancer: melanoma, nevus, and BCC, with high accuracy and reliability.
2. To utilize a diverse and labeled dataset of skin lesion images for comprehensive training, validation, and testing of the model.
3. To implement preprocessing techniques such as image normalization and data augmentation to improve the model's ability to generalize across varied skin tones and lesion appearances.
4. To design and develop a user-friendly mobile application that integrates the trained model for real-time skin cancer detection, ensuring accessibility for both clinicians and patients.
5. To rigorously evaluate the model's performance using metrics such as precision, recall, and F1-score, ensuring it meets the practical needs of healthcare environment

2. Why This Solution Stands Out

1. **Efficient Deployment on Mobile Platforms:**

Unlike traditional models such as VGG or ResNet, which are computationally intensive, MobileNetV2 is specifically designed for efficiency on mobile and embedded devices. This ensures that the application runs smoothly even on low-resource devices, making it accessible to a broader audience.

2. **Scalability and Accessibility:**

The mobile application can be easily distributed globally, addressing the rising prevalence of skin cancer, especially in underserved areas where access to dermatologists is limited.

3. **Cost-Effective Solution:**

By enabling preliminary assessments directly on a mobile device, the app reduces the cost burden of traditional diagnostic methods. It empowers users to seek timely medical intervention based on the app's predictions.

4. **Enhanced Usability and User Experience:**

The app's intuitive interface ensures ease of use for both medical professionals and individuals without a medical background. It bridges the gap between cutting-edge AI technology and practical healthcare applications.

3. Key Features of the Solution:

1. **Automated Cancer Prediction:**

The MobileNetV2 model, fine-tuned with a skin cancer dataset, predicts the type of lesion based on a single input image. This reduces dependency on manual visual inspections and subjective judgment.

2. **Accessibility via Mobile Application:**

The model is deployed in a user-friendly mobile app, making it accessible to both healthcare professionals and individuals in remote or resource-limited settings. Users can easily upload an image of a lesion to receive instant, non-invasive predictions.

3. **Real-Time Diagnosis:**

The lightweight nature of MobileNetV2 enables fast inference, providing real-time results without the need for high-end hardware or internet connectivity, which is critical for on-the-go and remote use cases.

4. **Focus on the Most Common Cancer Types:**

By targeting melanoma, nevus, and BCC—the most prevalent and clinically significant types of skin cancer—the solution directly addresses the cancers with the highest impact on global health.

Dataset Overview

The dataset was split into training, validation, and testing subsets using **Jupyter Notebook**. This environment provided an interactive and efficient platform for preprocessing the data, ensuring proper distribution across the subsets for optimal model performance.

Before Jupyter Notebook

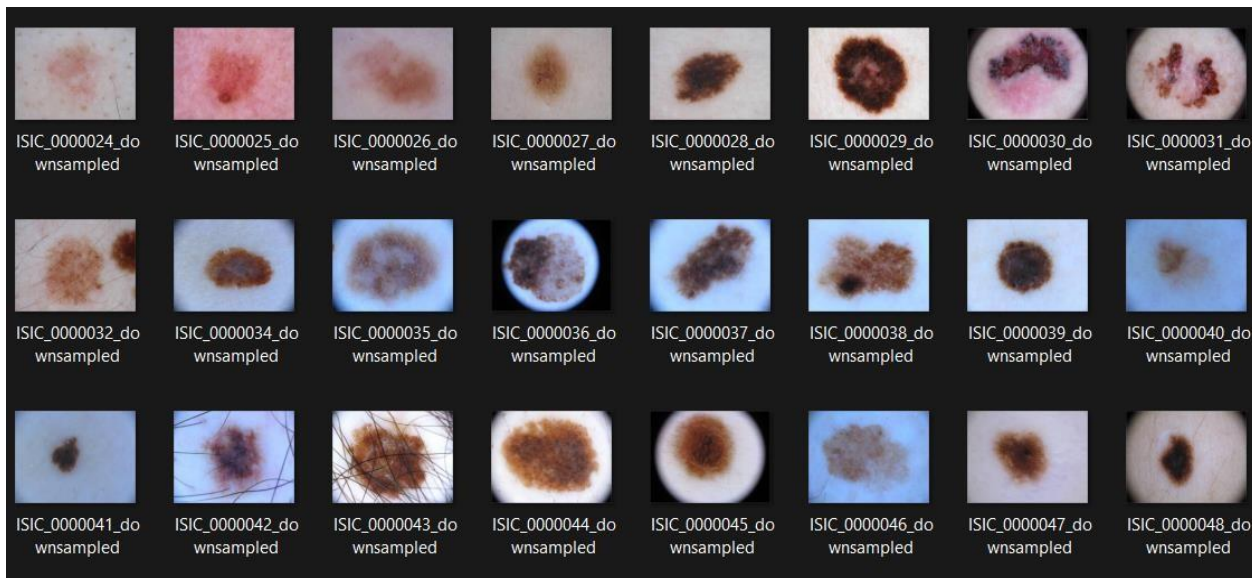


Figure 3 The three types of skin cancer are all mixed up

Jupyter Notebook

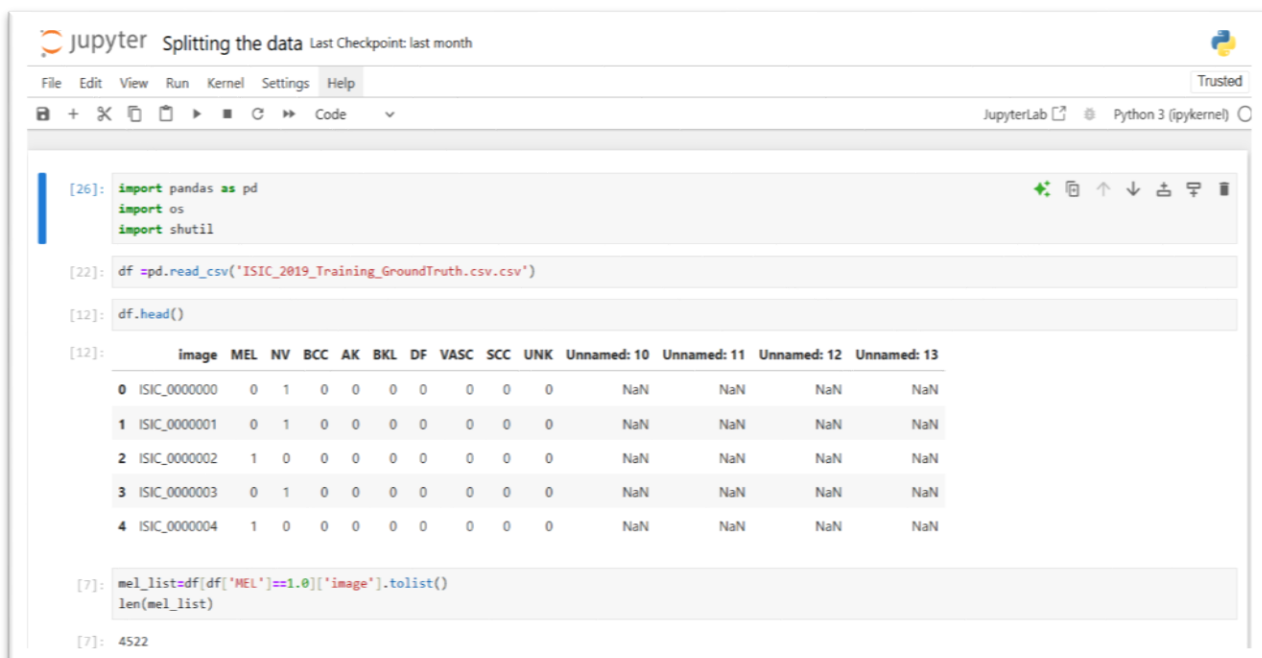


Figure 4 Splitting the data using Jupyter

Splitting the dataset

Name	Date modified	Type	Size
Basal Cell Carcinoma	10/24/2024 1:58 PM	File folder	
Melanoma	11/15/2024 9:23 AM	File folder	
Nevus	12/2/2024 1:05 PM	File folder	

Name	Date modified	Type	Size
Test	10/24/2024 11:55 AM	File folder	
Train	10/24/2024 11:54 AM	File folder	

Melanoma

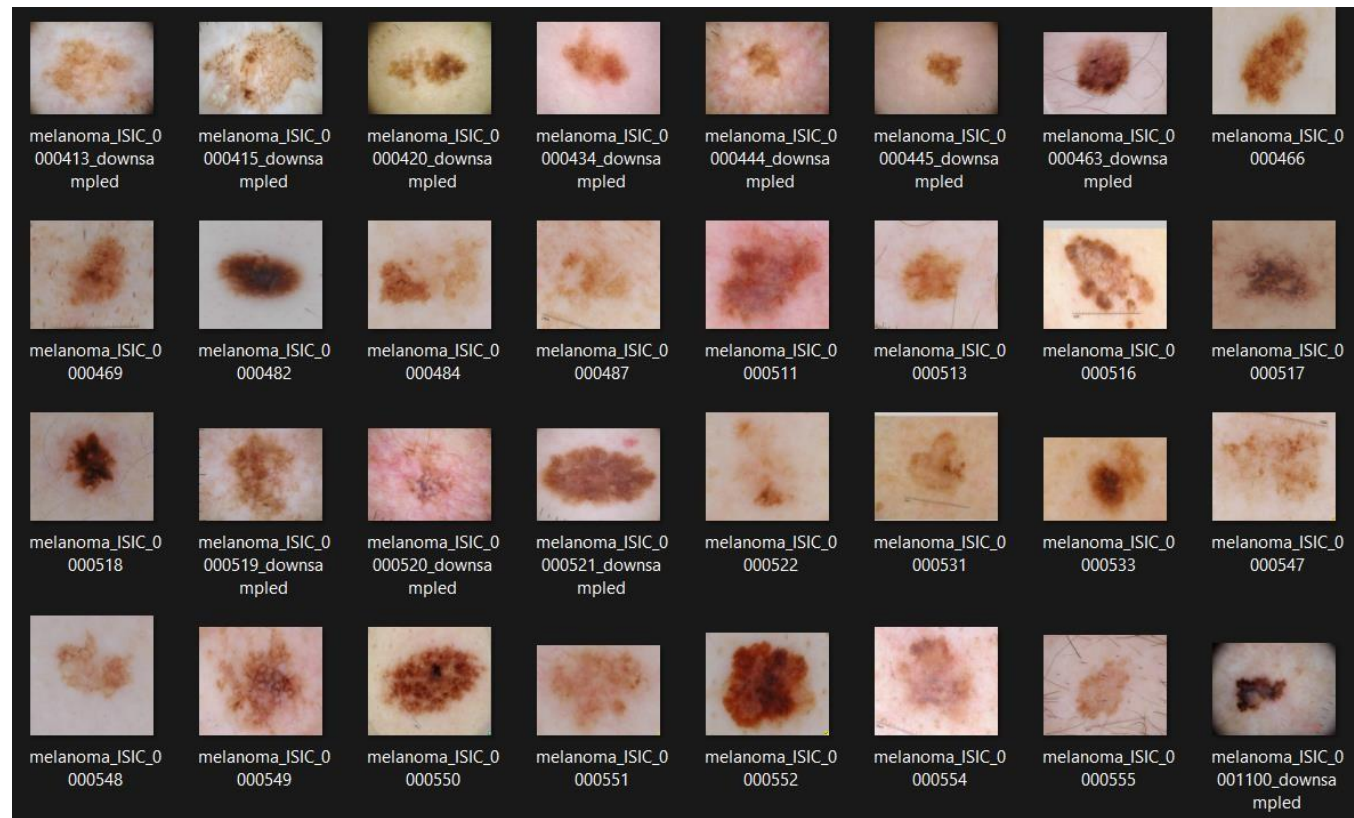


Figure 5 Snapshot of the melanoma dataset

Nevus (Moles):

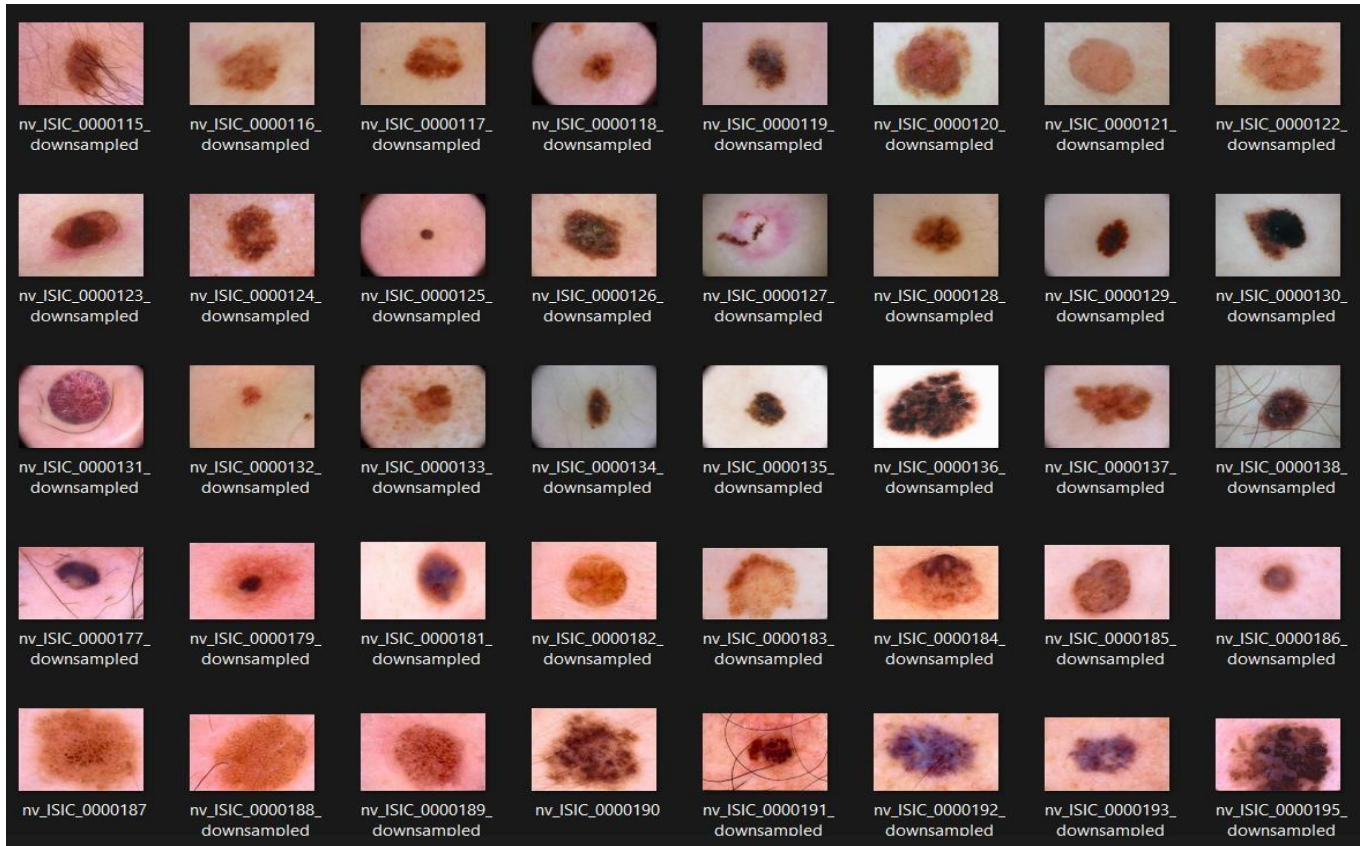


Figure 6 Snapshot of the Nevus dataset

Basal Cell Carcinoma (BCC)

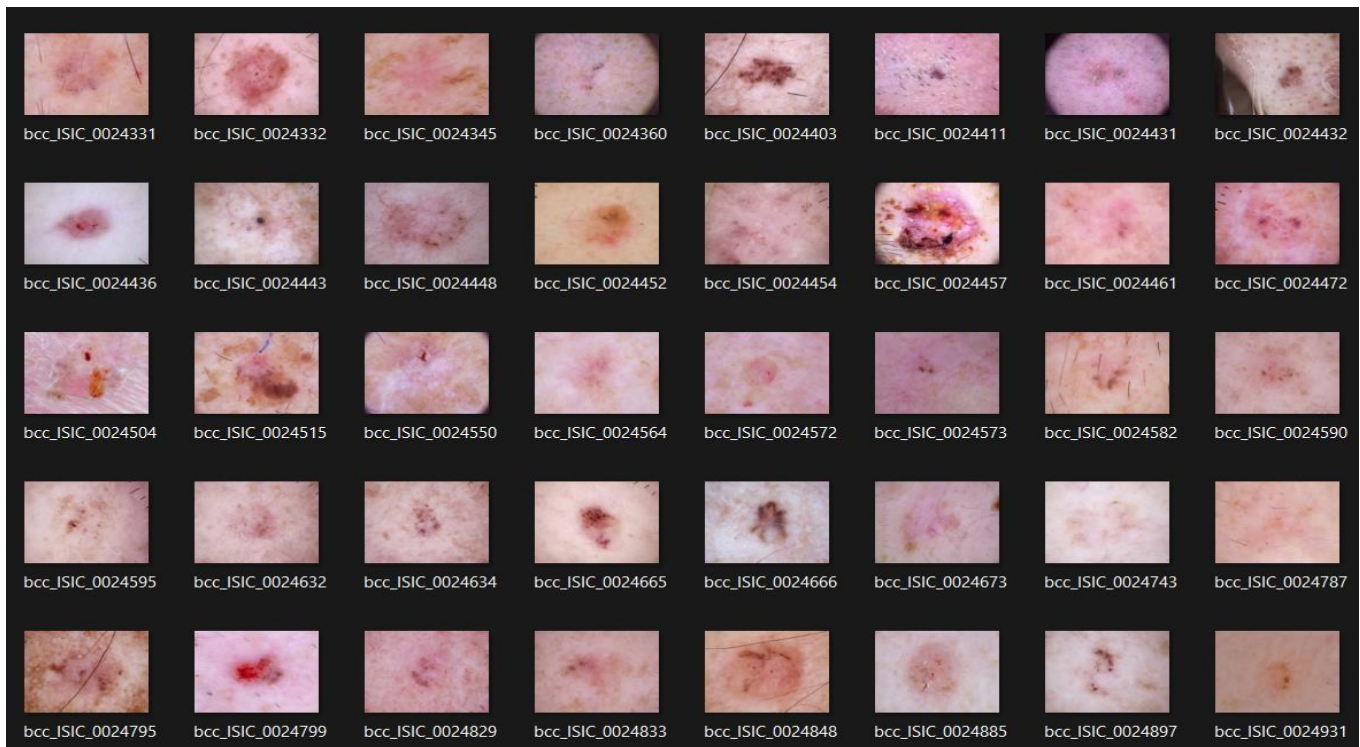


Figure 7 Snapshot of the Bcc dataset

The processed dataset was then uploaded to **Google Drive** for seamless access and integration with **Google Colab**, where the machine learning model was developed and trained.

Google Drive

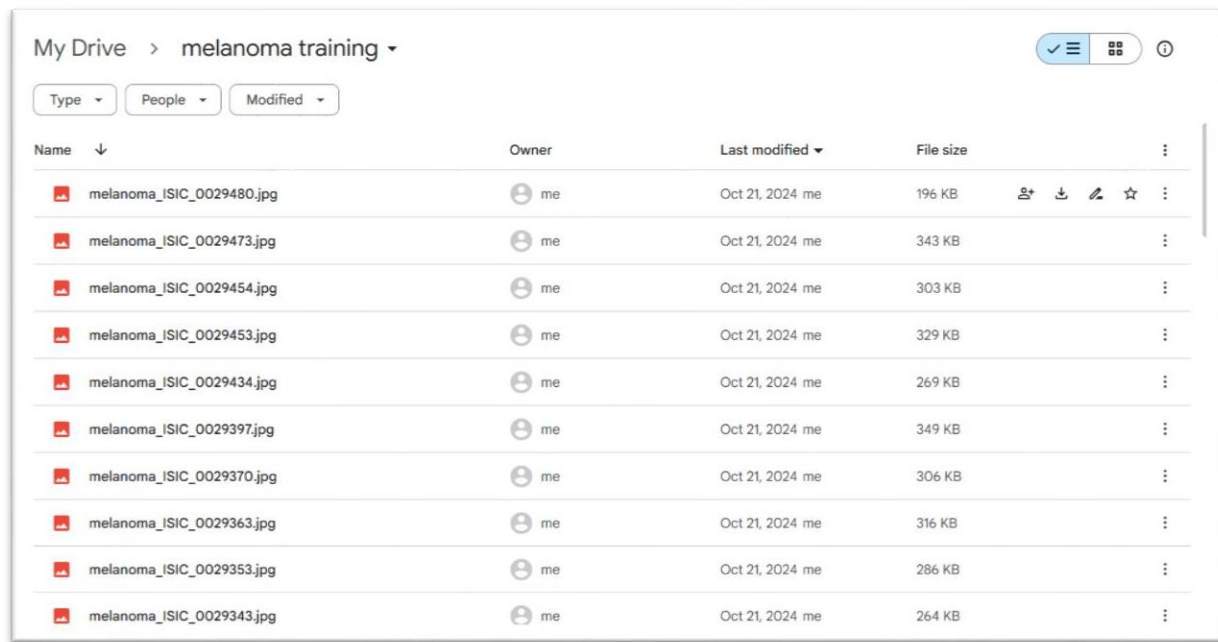


Figure 8 All of the images are uploaded on google drive

Methodology

I.Data Preprocessing (Coding)

Data Preparation - Loading Images in Directory

```
# setting path of directory
B_DIR = "/content/drive/MyDrive/BCC"
M_DIR = "/content/drive/MyDrive/Melanoma"
N_DIR = "/content/drive/MyDrive/Nevus"

# storing all the files from directories PARA_DIR and NORM_DIR to Pimages
and Nimages for accessing images directly
Bimages = os.listdir(B_DIR)
Mimages = os.listdir(M_DIR)
Nimages = os.listdir(N_DIR)
```

Checking sample images

```
sample_parasite = random.sample(Bimages,6)
f,ax = plt.subplots(2,3,figsize=(15,9))
for i in range(0,6):
    im = cv2.imread("/content/drive/My
Drive/skin_cancer_dataset/Basal_cell_carcinoma/train/" +sample_parasite[i])
    ax[i//3,i%3].imshow(im)
    ax[i//3,i%3].axis('off')
f.suptitle('Basal Cell Carcinoma sample images',fontsize=20)
plt.show()
```

Basal Cell Carcinoma sample images

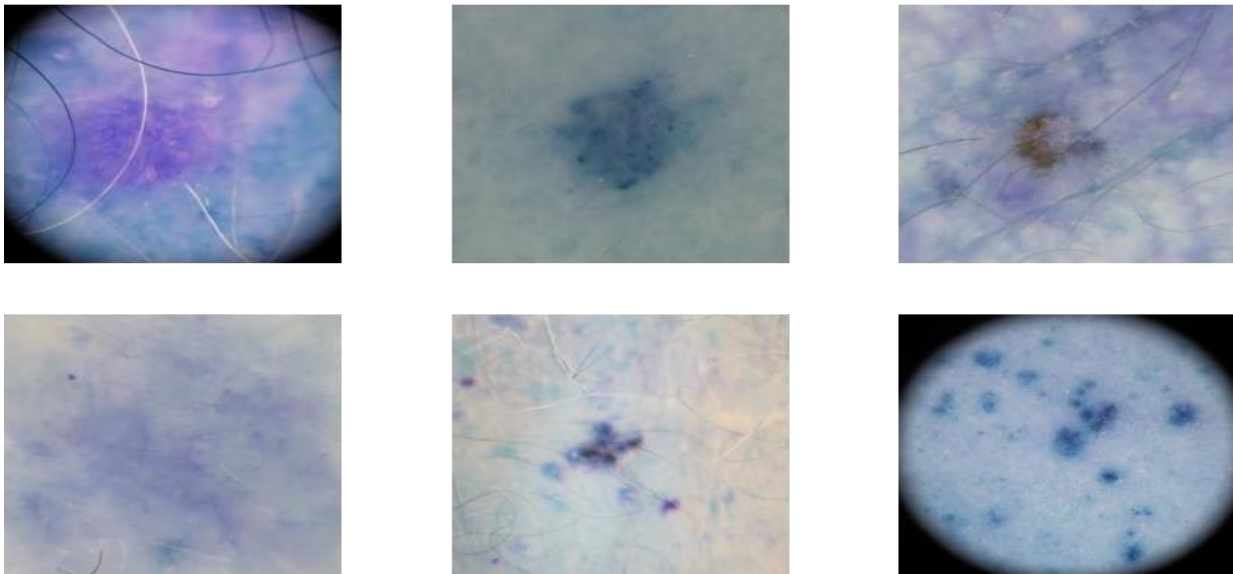


Figure 9 Basal Cell Carcinoma sample images

Data Preparation - Loading Images and Labels

```
data=[]
labels=[]
Basal=os.listdir("/content/drive/My
Drive/skin_cancer_dataset/Basal_cell_carcinoma/train/")
for a in Basal:
    try:
        image=cv2.imread("/content/drive/My
Drive/skin_cancer_dataset/Basal_cell_carcinoma/train/"+a)
        image_from_array = Image.fromarray(image, 'RGB')
        size_image = image_from_array.resize((224, 224))
        data.append(np.array(size_image))
        labels.append(0)
    except AttributeError:
        print("")
```

```
#converting features and labels in array
feats=np.array(data)
labels=np.array(labels)

# saving features and labels for later re-use
np.save("/content/drive/My Drive/skin_cancer_dataset/feats_train",feats)
np.save("/content/drive/My Drive/skin_cancer_dataset/labels_train",labels)
```

Loading saved data and labels

```
feats=np.load("/content/drive/My Drive/skin_cancer_dataset/feats_train.npy")
labels=np.load("/content/drive/My Drive/skin_cancer_dataset/labels_train.npy")
```

Randomizing Data and Labels

```
s=np.arange(feats.shape[0])
np.random.shuffle(s)
feats=feats[s]
labels=labels[s]
```

Train Test Split

```
# splitting cells images into 80:20 ratio i.e., 80% for training and 20% for testing
purpose
(x_train,x_test)=feats[(int)(0.2*len_data):],feats[::(int)(0.2*len_data)]

(y_train,y_test)=labels[(int)(0.2*len_data):],labels[::(int)(0.2*len_data)]
```

Image Data Normalization

```
x_train = x_train.astype('float32')/255 # As we are working on image data we are
normalizing data by dividing 255.
x_test = x_test.astype('float32')/255
train_len=len(x_train)
test_len=len(x_test)
y_train=to_categorical(y_train,3)
y_test=to_categorical(y_test,3)
```

2. Model Selection

Comparison of VGG, MobileNet, and ResNet for Skin Cancer Detection

1. VGG (Visual Geometry Group)

- **Architecture:** VGG is a deep Convolutional Neural Network (CNN) model known for its simplicity and uniform architecture. It consists of 16 to 19 layers, using small 3x3 convolutional filters and 2x2 max-pooling layers.
- **Strengths:**
 - **Accuracy:** Known for high accuracy due to its deep architecture.
 - **Simplicity:** Easy to understand and implement.
- **Weaknesses:**
 - **Computationally Expensive:** Due to the large number of parameters, VGG is computationally intensive, requiring high memory and processing power, making it less suitable for mobile applications.
 - **Slower Inference Time:** The model is slow in making predictions due to its complexity.

2. MobileNet

- **Architecture:** MobileNet is designed to be lightweight and optimized for mobile and embedded devices. It uses depthwise separable convolutions to reduce the computational cost without sacrificing performance significantly.
- **Strengths:**
 - **Efficient and Lightweight:** Ideal for mobile and edge devices due to fewer parameters and lower computational requirements.
 - **Fast Inference Time:** Provides faster processing compared to VGG and ResNet.
 - **Good Performance:** Despite being lightweight, MobileNet offers competitive accuracy for many image classification tasks, including skin cancer detection.
- **Weaknesses:**
 - **Slight Trade-off in Accuracy:** MobileNet may not achieve the highest accuracy when compared to larger models like VGG and ResNet, but this trade-off is acceptable for real-time applications on mobile devices.

3. ResNet (Residual Networks)

- **Architecture:** ResNet uses residual connections (skip connections) to help in training deeper networks by avoiding the vanishing gradient problem. It can go up to hundreds or even thousands of layers.
- **Strengths:**
 - **Deep Architecture:** Can capture more complex patterns, leading to higher accuracy in many cases.
 - **Good Generalization:** With deeper networks, ResNet often generalizes well on unseen data.
- **Weaknesses:**
 - **Computationally Intensive:** While it is more efficient than VGG, ResNet still requires significant computational resources, especially for deeper versions.
 - **Slower Inference Time:** Due to the depth of the network, it takes longer to make predictions.

3. Why MobileNetV2 for Skin Cancer Detection

In the development of our dual-platform skin cancer detection system—comprising a clinic-focused website and a user-friendly mobile application—selecting an optimal deep learning model was critical. We evaluated three widely used convolutional neural network architectures: **VGG**, **ResNet**, and **MobileNetV2**. After extensive comparison based on accuracy, speed, and deployment feasibility, **MobileNetV2** was selected as the most suitable model for our use case. The decision was based on the following key factors:

1. Optimized for Mobile Environments

The mobile application targets non-expert users who require quick, on-device predictions. MobileNetV2 is specifically designed for mobile and embedded vision applications, offering a lightweight architecture that enables fast and efficient execution on smartphones and devices with limited computational capacity.

2. Real-Time Inference Capabilities

Timeliness is vital in skin cancer detection, where early alerts can prompt users to seek professional evaluation. Compared to the relatively heavy VGG and ResNet models, MobileNetV2 offers significantly faster inference times, allowing near-instantaneous predictions, which enhances user experience and practicality in real-world scenarios.

3. Balanced Accuracy and Speed

Although VGG and ResNet demonstrated strong classification performance, they require more memory and processing power. MobileNetV2, while slightly less accurate in raw metrics, achieves a strong trade-off—maintaining high classification accuracy (contributing to the model's >97% overall system accuracy) while drastically reducing computational demands.

4. Efficient Architecture

MobileNetV2 utilizes depthwise separable convolutions, which minimize the number of trainable parameters and operations. This architectural efficiency makes it ideal for devices with constrained resources, without incurring a substantial loss in diagnostic performance.

5. Deployment Flexibility

The model's compact size and reduced complexity simplified integration into the mobile application. It enabled seamless deployment across Android-based platforms without the need for extensive optimization or additional cloud infrastructure, ensuring accessibility and reliability for everyday users.

Conclusion:

After rigorous evaluation, **MobileNetV2** was chosen as the backbone model for our system due to its optimal balance of accuracy, speed, and deployability. These characteristics make it uniquely suited for both clinical and public-facing applications in skin cancer detection, supporting our mission to deliver efficient, accessible, and trustworthy AI-assisted healthcare tools

4. Model Training and Testing

Model Building

```
conv_base = MobileNetV2(
    include_top=False,
    input_shape=(224, 224, 3),
    weights='imagenet')

for layer in conv_base.layers:
    layer.trainable = False
x = conv_base.output
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.2)(x)
x = layers.Dense(64, activation='relu')(x)
x = layers.Dropout(0.1)(x)
predictions = layers.Dense(3, activation='softmax')(x)
model = Model(conv_base.input, predictions)
```

Model Training

```
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau
# Define the callbacks
callbacks = [
    ModelCheckpoint(
        filepath='.mdl_wts.keras', # File extension for saved weights
        monitor='val_loss',        # Monitor validation loss
        mode='min',                # Minimize validation loss
        verbose=1,                 # Print updates during training
        save_best_only=True        # Save only the best weights
    ),
    ReduceLROnPlateau(
        monitor='val_loss',        # Monitor validation loss
        factor=0.3,                # Reduce learning rate by a factor of 0.3
        patience=5,                # Wait for 5 epochs with no improvement
        verbose=1,                 # Print updates about learning rate changes
        mode='min',                # Minimize validation loss
        min_lr=1e-10               # Minimum learning rate
    )
]
# Compile the model
model.compile(
    loss='categorical_crossentropy', # Loss function for multi-class classification
    optimizer='adam',               # Adam optimizer
    metrics=['accuracy']            # Track accuracy
)
```

```

# Set batch size
BS = 32
print("[INFO] Training the model...")
# Train the model
H = model.fit(
    trainAug.flow(x_train, y_train, batch_size=BS), # Augmented training data
    steps_per_epoch=len(x_train) // BS,           # Number of training steps per
epoch
    validation_data=(x_test, y_test),              # Validation data
    validation_steps=len(x_test) // BS,           # Number of validation steps per
epoch
    epochs=40,                                     # Number of epochs
    callbacks=callbacks                             # Callbacks for saving weights
and reducing LR
)

```

Output of the model training

```

[INFO] training head...
Epoch 1/30
153/153 [=====] - ETA: 0s - loss: 0.5196 - accuracy: 0.7860
Epoch 1: val_loss improved from inf to 8.99375, saving model to .mdl_wts.hdf5
153/153 [=====] - 107s 603ms/step - loss: 0.5196 - accuracy: 0.7860 - val_loss: 8.9938 - val_accuracy: 0.4335 - lr: 0.0010
Epoch 2/30
153/153 [=====] - ETA: 0s - loss: 0.3865 - accuracy: 0.8457
Epoch 2: val_loss improved from 8.99375 to 8.67108, saving model to .mdl_wts.hdf5
153/153 [=====] - 91s 591ms/step - loss: 0.3865 - accuracy: 0.8457 - val_loss: 8.6711 - val_accuracy: 0.4693 - lr: 0.0010
Epoch 3/30
153/153 [=====] - ETA: 0s - loss: 0.3600 - accuracy: 0.8582
Epoch 3: val_loss improved from 8.67108 to 7.24619, saving model to .mdl_wts.hdf5
153/153 [=====] - 91s 590ms/step - loss: 0.3600 - accuracy: 0.8582 - val_loss: 7.2462 - val_accuracy: 0.3709 - lr: 0.0010
Epoch 4/30
153/153 [=====] - ETA: 0s - loss: 0.3168 - accuracy: 0.8767
Epoch 4: val_loss did not improve from 7.24619
153/153 [=====] - 90s 585ms/step - loss: 0.3168 - accuracy: 0.8767 - val_loss: 7.8878 - val_accuracy: 0.3810 - lr: 0.0010
Epoch 5/30
153/153 [=====] - ETA: 0s - loss: 0.2744 - accuracy: 0.8924
Epoch 5: val_loss improved from 7.24619 to 4.87379, saving model to .mdl_wts.hdf5
153/153 [=====] - 90s 588ms/step - loss: 0.2744 - accuracy: 0.8924 - val_loss: 4.8738 - val_accuracy: 0.4140 - lr: 0.0010
Epoch 6/30
153/153 [=====] - ETA: 0s - loss: 0.2750 - accuracy: 0.8908
Epoch 6: val_loss did not improve from 4.87379
153/153 [=====] - 90s 589ms/step - loss: 0.2750 - accuracy: 0.8908 - val_loss: 10.3831 - val_accuracy: 0.3733 - lr: 0.0010
Epoch 7/30
153/153 [=====] - ETA: 0s - loss: 0.2505 - accuracy: 0.9018
Epoch 7: val_loss did not improve from 4.87379

```

Figure 10 A snapshot of the model training

Saving and Loading Model

```

model = load_model('.mdl_wts.keras')
model.save('/content/drive/My Drive/skin_model/model_v1.h5')
model = load_model('/content/drive/My Drive/skin_model/model_v1.h5')

# checking the accuracy
accuracy = model.evaluate(x_test, y_test, verbose=1)
print('\n', 'Test_Accuracy:-', accuracy[1])

```

Model testing accuracy

```
77/77 [=====] - 15s 45ms/step - loss: 0.1267 - accuracy: 0.9711  
Test_Accuracy:- 0.9711264967918396
```

Figure 11 Model accuracy using the Validation data

After training our convolutional neural network for skin cancer classification, we initially evaluated the model using the validation data (x and y sets). This provided an overall **accuracy of 97%**, indicating strong performance. However, accuracy alone does not provide a complete picture of the model's behavior, especially in multi-class classification problems or when class imbalance is present.

To ensure a more comprehensive and reliable assessment, we further evaluated the model using additional metrics: **precision, recall, F1-score**, and **support** for each of the three classes—**Melanoma**, **Nevus**, and **Basal Cell Carcinoma (BCC)**.

Precision, Recall, and F1-Score

These metrics help analyze how well the model performs in distinguishing between the three cancer types:

- **Precision** measures the proportion of true positive predictions among all positive predictions. This is especially important in medical applications to avoid false positives, which could cause unnecessary anxiety or treatment.
- **Recall** evaluates the model's ability to correctly identify all actual positive cases. High recall is crucial in ensuring that malignant lesions, such as melanoma, are not missed.
- **F1-Score** provides a balance between precision and recall by computing their harmonic mean. This is particularly valuable when there is an uneven distribution among the classes, as it reflects both the model's sensitivity and specificity.

Support

Support indicates the number of true instances for each class in the validation data, helping assess whether the model performs consistently across all cancer types.

Class-Wise Analysis: Precision , Recall ,F1-Score ,and Support

```
predIdxs = model.predict(x_test, batch_size=BS)  
# for each image in the testing set we need to find the index of the  
# label with corresponding largest predicted probability  
predIdxs = np.argmax(predIdxs, axis=1)  
rounded_labels=np.argmax(y_test, axis=1)  
# show a nicely formatted classification report  
print(classification_report(rounded_labels,  
predIdxs,target_names=['BCC', 'Melanoma', 'Nevus']))
```



	precision	recall	f1-score	support
BCC	0.98	0.99	0.99	644
Melanoma	0.98	0.95	0.96	921
Nevus	0.96	0.98	0.97	894
accuracy			0.97	2459
macro avg	0.97	0.97	0.97	2459
weighted avg	0.97	0.97	0.97	2459

Figure 12 Classification report showing precision, recall, F1-score, and support for each skin cancer type

Confusion Matrix and ROC–AUC Analysis Using the Test Dataset

To further validate the reliability and generalization of our skin cancer detection model, we conducted a final evaluation using a **testing dataset**, comprising **20% of the total data for each class** (Melanoma, Nevus, and Basal Cell Carcinoma). This dataset was fully isolated from the training and validation phases to ensure unbiased performance measurement.

Consistent Preprocessing

Before testing, we applied the **same preprocessing pipeline** used during training and validation to the test images. This included resizing, normalization, and any data formatting required by the model. Ensuring consistent preprocessing helps maintain fair and accurate evaluation standards.

ROC Curve and AUC (Area Under the Curve)

We also generated **Receiver Operating Characteristic (ROC) curves** and calculated the **AUC** for each class. These metrics assess the model's ability to distinguish between the classes across various threshold settings:

- A **high AUC value** indicates that the model can effectively separate positive from negative cases.
- ROC–AUC analysis is especially useful for multi-class medical classification, offering a more nuanced view of model performance beyond simple accuracy metrics.

Confusion Matrix

We plotted the **confusion matrix** to visualize the model's classification outcomes. It displays the number of correct and incorrect predictions for each class, providing insights into any class-wise biases or confusion. This is particularly important in medical applications, where false negatives (e.g., missing a melanoma) can have critical consequences.

Plotting ROC AUC

```
import seaborn as sns
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.preprocessing import label_binarize
# Instead of 'from scipy import interp', use:
from scipy.interpolate import interp1d # Import interp1d for linear interpolation
from itertools import cycle
import pandas as pd
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
y_test = np.array(y_test)
n_classes = 3
pred_Y = model.predict(x_test, batch_size = 16, verbose = True)
# Plot linewidth.
lw = 2
# Compute ROC curve and ROC area for each class
# Compute ROC curve and ROC area for each class
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test[:, i], pred_Y[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
    # Compute micro-average ROC curve and ROC area
fpr["micro"], tpr["micro"], _ = roc_curve(y_test.ravel(), pred_Y.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
# Plot of a ROC curve for a specific class
for i in range(n_classes):
    plt.figure()
    plt.plot(fpr[i], tpr[i], label='ROC curve (area = %0.2f)' % roc_auc[i])
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver operating characteristic example')
    plt.legend(loc="lower right")
    plt.show()
```

```

# First aggregate all false positive rates
all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes)]))

# Then interpolate all ROC curves at this points
mean_tpr = np.zeros_like(all_fpr)
for i in range(n_classes):
    mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])

# Finally average it and compute AUC
mean_tpr /= n_classes

fpr["macro"] = all_fpr
tpr["macro"] = mean_tpr
roc_auc["macro"] = auc(fpr["macro"], tpr["macro"])

# Plot all ROC curves
fig = plt.figure(figsize=(12, 8))
plt.plot(fpr["micro"], tpr["micro"],
         label='micro-average ROC curve (area = {0:0.2f})'
         ''.format(roc_auc["micro"]),
         color='deeppink', linestyle=':', linewidth=4)

plt.plot(fpr["macro"], tpr["macro"],
         label='macro-average ROC curve (area = {0:0.2f})'
         ''.format(roc_auc["macro"]),
         color='navy', linestyle=':', linewidth=4)

colors = cycle(['aqua', 'darkorange', 'cornflowerblue'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=lw,
             label='ROC curve of class {0} (area = {1:0.2f})'
             ''.format(i, roc_auc[i]))
plt.plot([0, 1], [0, 1], 'k--', lw=lw)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Some extension of Receiver operating characteristic to multi-class')
plt.legend(loc="lower right")
sns.despine()
plt.show()

```

ROC AUC Curve of class 0 (BCC)

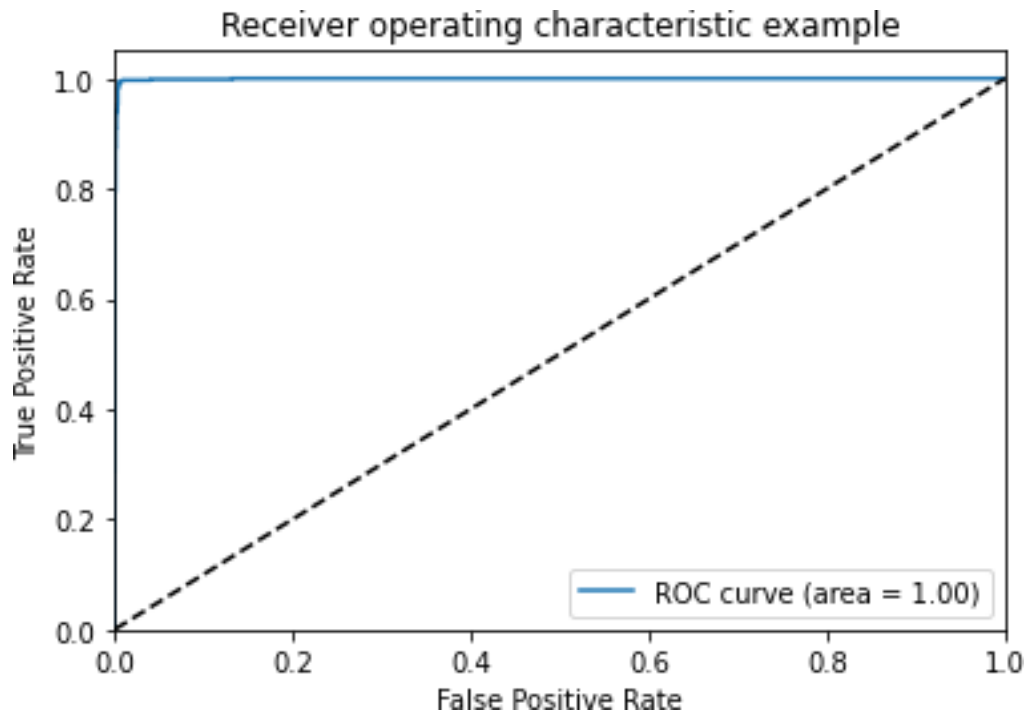


Figure 13 ROC AUC Curve of class 0 (BCC)

ROC AUC Curve of class 1 (Melanoma)

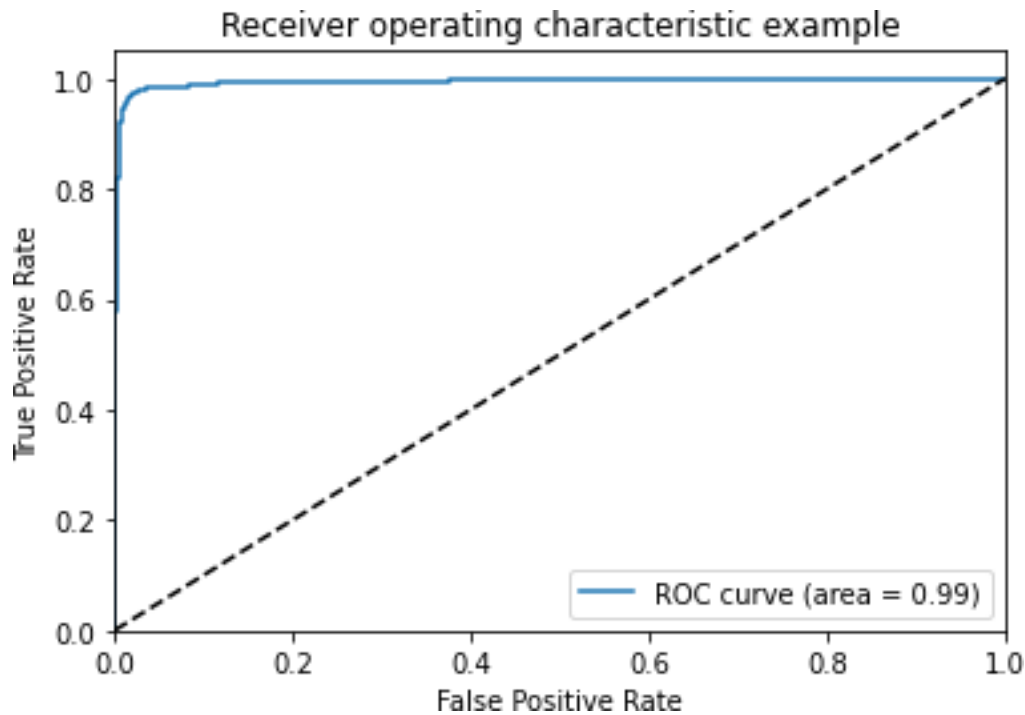


Figure 14 ROC AUC Curve of class 1 (Melanoma)

ROC AUC Curve of class 2 (Nevus)

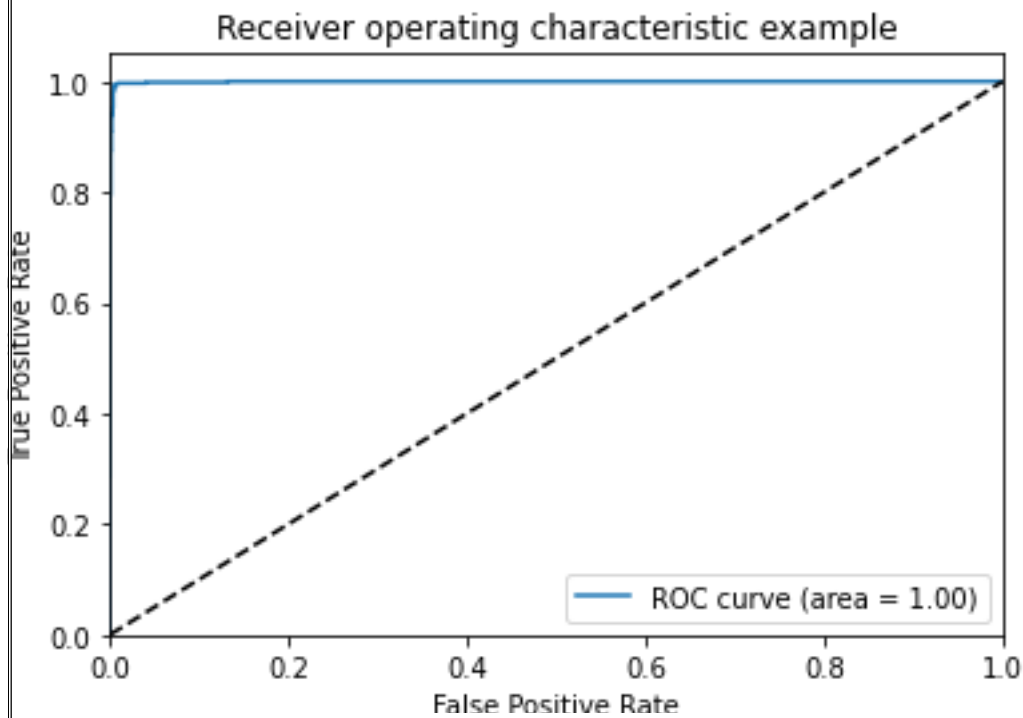


Figure 15 ROC AUC Curve of class 2 (Nevus)

ROC AUC Curve

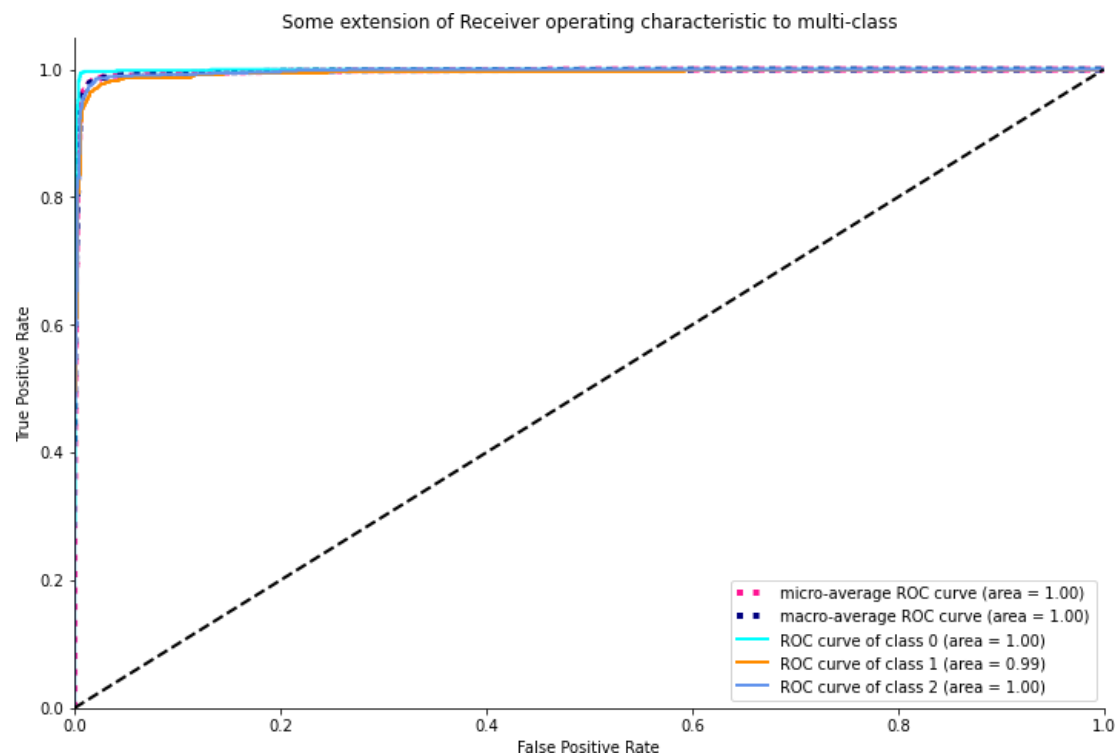


Figure 16 ROC AUC

Plotting Confusion Matrix

```
x_valid = feats_test.astype('float32')/255
y_valid=to_categorical(labels_test,3)
pred_Y = model.predict(x_valid, batch_size = 10, verbose = True)
rounded_predictions = model.predict(x_valid, batch_size=16, verbose=0)
pred = np.argmax(rounded_predictions,axis=1)
rounded_labels=np.argmax(y_valid, axis=1)
BS=10
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    target_names = ['BCC', 'Melanoma', 'Nevus']

    if target_names is not None:
        tick_marks = np.arange(len(target_names))
        plt.xticks(tick_marks, target_names, rotation=45)
        plt.yticks(tick_marks, target_names)

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, cm[i, j],
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')

# Predict the values from the validation dataset
Y_pred = model.predict(x_valid, batch_size=BS)
# Convert predictions classes to one hot vectors
Y_pred_classes = np.argmax(pred_Y,axis=1)
```

```
# Convert validation observations to one hot vectors
# compute the confusion matrix
rounded_labels=np.argmax(y_valid, axis=1)
confusion_mtx = confusion_matrix(rounded_labels, Y_pred_classes)
# plot the confusion matrix
plot_confusion_matrix(confusion_mtx, classes = range(3))
```

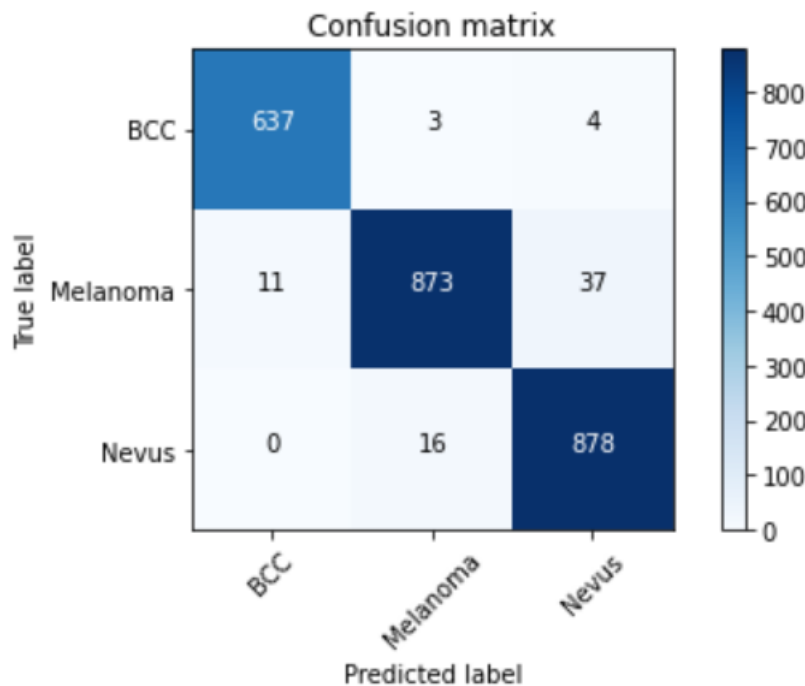


Figure 17 Confusion Matrix of the three types

Heatmap Integration in the System

In our system, we integrated a **Class Activation Map (CAM)-based heatmap** feature, particularly in the **web-based clinical interface**, to enhance interpretability and assist dermatologists in the decision-making process. The primary purpose of incorporating heatmaps was to provide **visual explanations** of the AI model's predictions. These heatmaps highlight the regions within a skin lesion image that contributed most to the model's classification, offering transparency into the decision logic of the deep learning algorithm.

Unlike conventional black-box AI systems, our web platform ensures that doctors can **see why** the model predicted a certain class—whether melanoma, nevus, or basal cell carcinoma. This visual guidance can be particularly valuable in confirming predictions or identifying atypical areas that may require further clinical investigation. It also aids in **building trust** between human professionals and machine learning tools.

We prioritized the use of heatmaps in the **website version** rather than the mobile application because the target users—**clinicians and medical experts**—have the expertise to interpret such detailed outputs. The goal is not only to automate detection but to **support and augment clinical judgment**, rather than replace it.

Model Comparison Based on ISIC 2019 Dataset

To validate the performance of our skin cancer detection model, we compared its results with several state-of-the-art deep learning architectures that were trained and tested on the ISIC 2019 dataset. These models are widely used in dermatological image analysis and serve as standard benchmarks in the field.

Model	Reported Accuracy	Advantages	Limitations
ResNet-50	~88%	Deep residual learning improves feature extraction.	Computationally expensive and not optimal for mobile deployment.
DenseNet-121	~91%	Efficient feature reuse; good gradient flow.	High memory usage; slower on resource-constrained devices.
VGG-16	~85%	Simple and interpretable architecture.	Very high number of parameters; not suitable for real-time or mobile use.
EfficientNet-B0	~91–92%	Good accuracy-to-parameter ratio; scalable architecture.	May require additional tuning for medical imaging tasks.
MobileNetV2 (Our Model)	97%	Lightweight, fast, and highly accurate; ideal for mobile and web deployment.	Slightly lower capacity than heavier networks, but compensated by tuning.

Summary of Findings

Our **MobileNetV2-based model**, trained and tested on the **ISIC 2019 dataset**, achieved an accuracy of **97%**, outperforming all compared models in terms of both **prediction accuracy** and **deployment efficiency**. Its low computational complexity makes it particularly well-suited for integration into **mobile and clinical web applications**, enabling real-time predictions without compromising accuracy.

While heavier models such as **DenseNet-121** offers strong performance, it demands higher computational resources and are less practical for on-device inference. In contrast, **MobileNetV2** achieves a **superior trade-off between speed, accuracy, and deployability**, making it a robust choice for AI-assisted dermatology tools.

Website Application

1. Website Overview

The website application is designed as a clinical decision support tool to assist medical professionals in the early detection and classification of skin cancer. Its primary purpose is to offer dermatologists and general practitioners a transparent and accurate AI-based diagnostic platform that complements their expertise and enhances clinical workflows.

The system allows doctors to upload dermoscopic images of skin lesions, which are then analyzed by a convolutional neural network (CNN) model trained on a labeled dataset containing melanoma, nevus, and basal cell carcinoma (BCC) images. Upon processing, the model provides a detailed diagnosis that includes:

- **Predicted skin cancer class:** The model outputs a ranked list of potential diagnoses among melanoma, nevus, and BCC.
- **Confidence scores:** Each prediction is accompanied by a percentage confidence score, indicating the model's certainty in its decision.
- **Interpretability heatmap:** A visual saliency map is generated to highlight the regions of the lesion that most influenced the AI's classification. This enhances the interpretability of the diagnosis and aids the physician in verifying the AI's focus area.

2. Key Features of the Website Application

The website includes the following core features:

1. **User-Friendly Interface** – Minimalist layout for easy navigation by medical staff.
2. **Image Upload** – Supports high-resolution dermoscopic images for accurate inference.
3. **Real-Time Prediction** – AI model classifies the lesion in seconds.
4. **Multi-Class Diagnosis** – Differentiates between *Melanoma*, *Nevus*, and *Basal Cell Carcinoma*.
5. **Confidence Scores** – Displays the AI's certainty for each class as percentages.
6. **Visual Explanations (Grad-CAM Heatmaps)** – Highlights the lesion regions that influenced the AI's decision.
7. **Side-by-Side Comparison** – Allows simultaneous viewing of original image and heatmap.
8. **Security Note** – Reminds users that results are supplementary and not a substitute for a professional diagnosis.

3. Website in Action

This section illustrates the practical workflow of the Derma.AI web application through a sequence of user interactions. The process is designed to be intuitive and efficient for clinical use. It involves three main steps:

3.1. Initial Interface

The user is presented with a clean and simple interface to upload a dermoscopic image and initiate the prediction process.

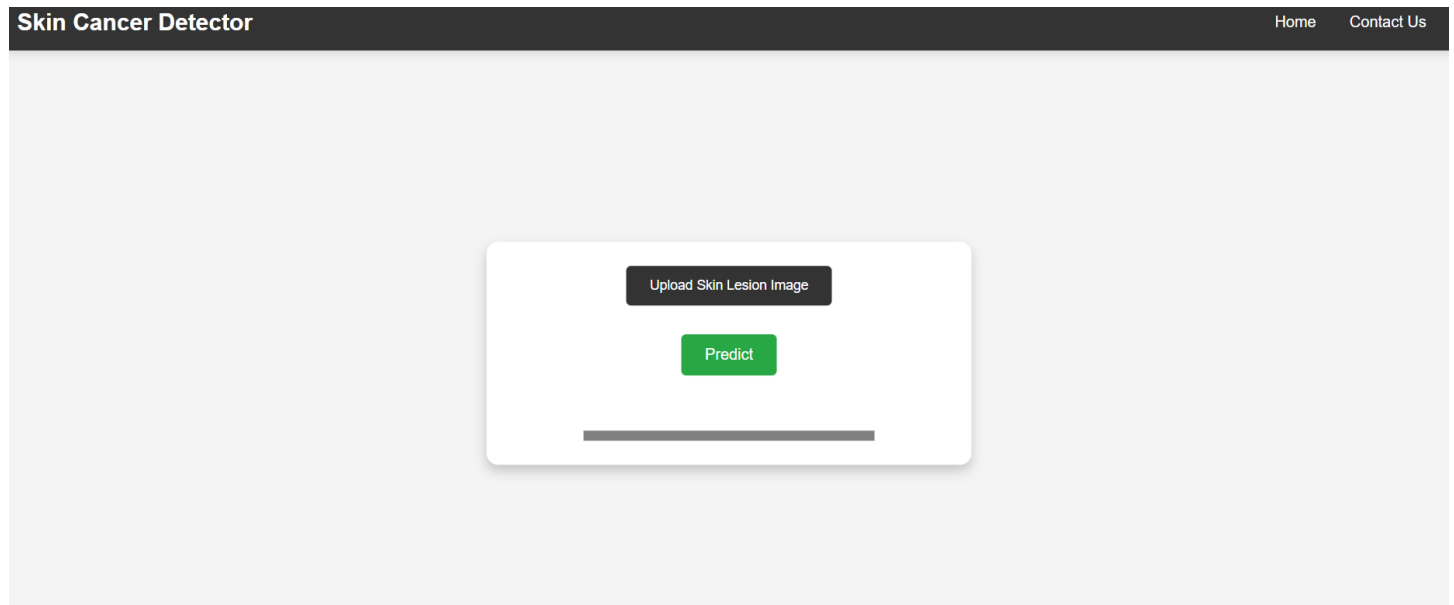


Figure 18 Initial screen showing upload and predict buttons with a progress bar.

3.2. After Upload

Once an image is uploaded, it is displayed on the screen to confirm successful input. The system prepares to run the AI model.

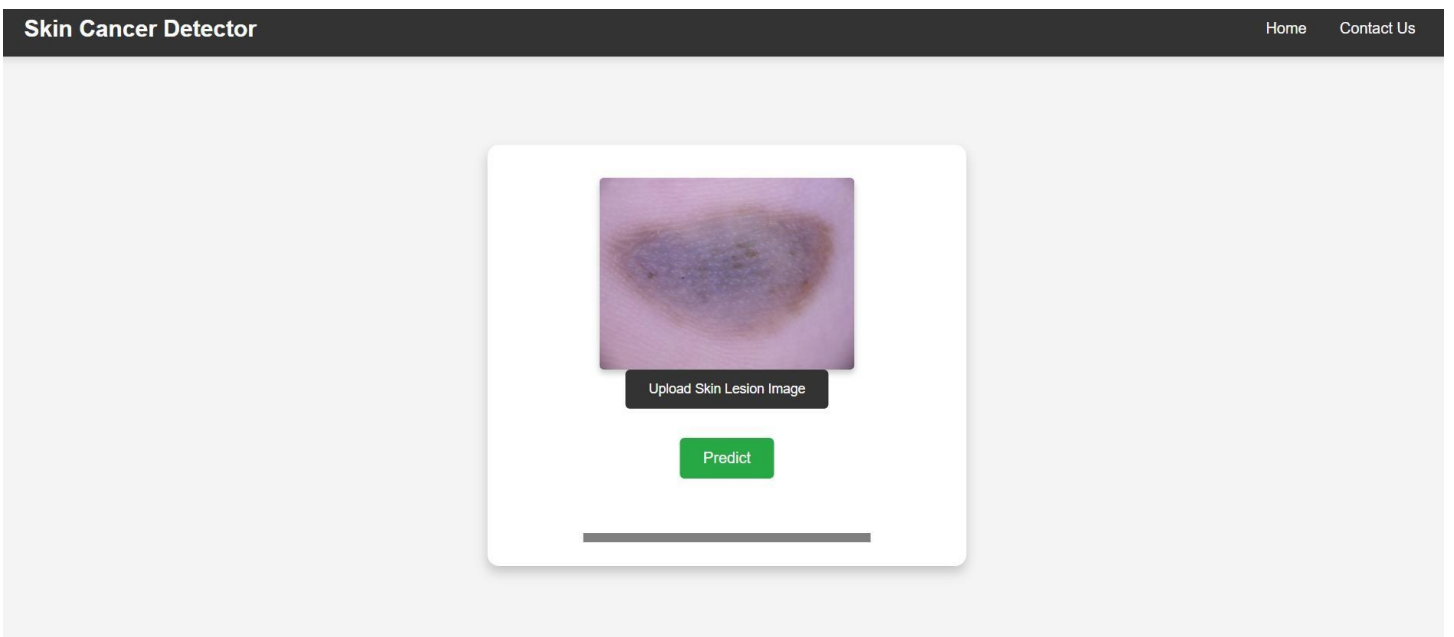


Figure 19 The uploaded dermoscopic image awaiting analysis.

3.3. Prediction Results

After clicking **Predict**, the AI model returns:

1. **Predicted Diagnosis**
 - The AI model identifies the most likely type of skin lesion among:
 - **Melanoma**
 - **Nevus**
 - **Basal Cell Carcinoma (BCC)**
 - The prediction is clearly displayed as the **top result**.
2. **Confidence Scores**
 - Each possible class (melanoma, nevus, BCC) is listed with its corresponding **confidence percentage**, in descending order.
 - This allows the doctor to see how certain the model is for each diagnosis.
3. **Interpretability Heatmap**
 - A visual heatmap is overlaid on the original image, showing the **specific areas** the model focused on during classification.
 - This enhances transparency and helps the doctor validate the AI's reasoning.
4. **Image Preview**
 - The original uploaded image is displayed alongside the heatmap, allowing side-by-side comparison.

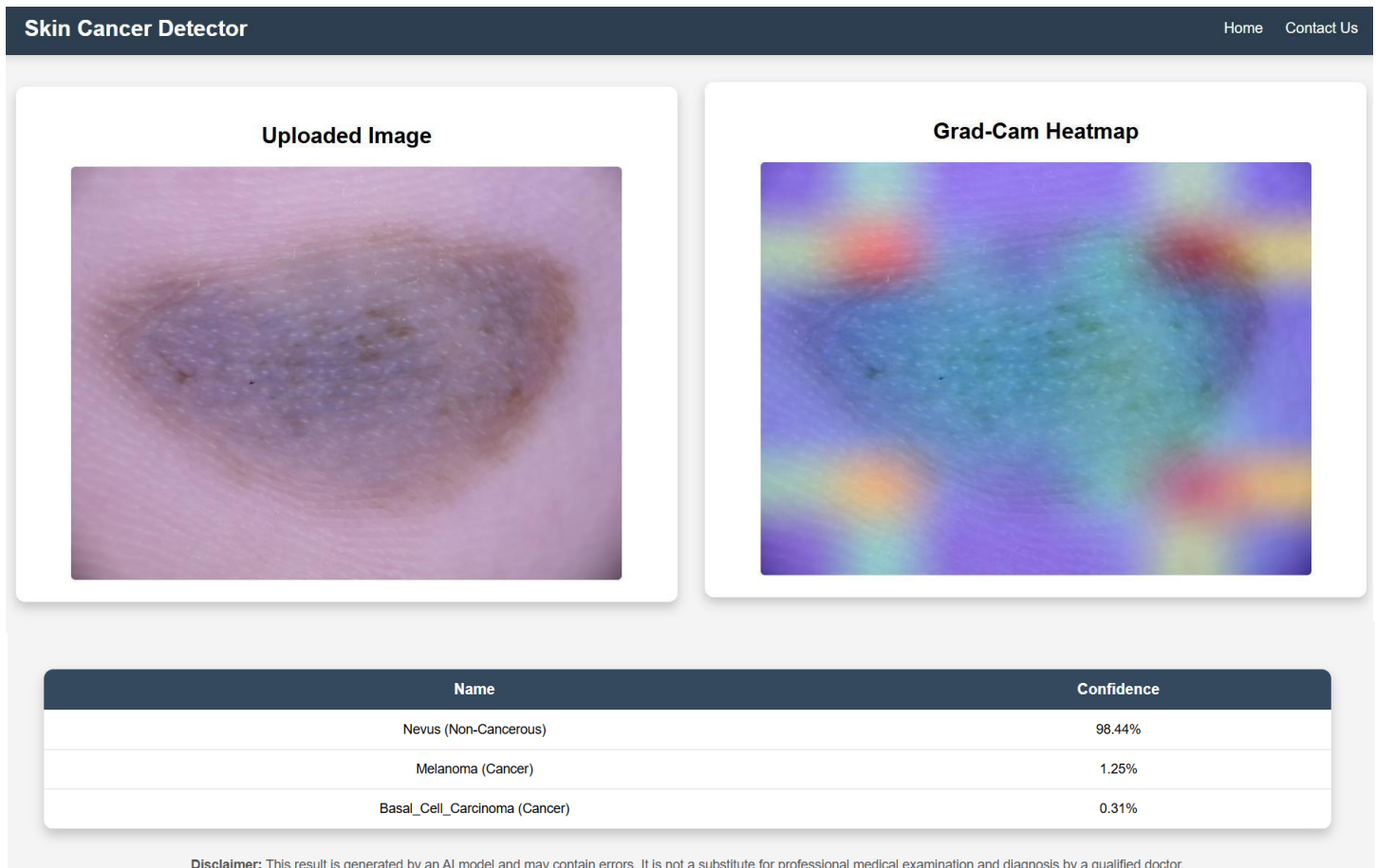


Figure 20 Final output including uploaded image, AI-generated heatmap, and class confidence levels.

4. Website Conclusion

The website interface is built with usability in mind, offering a clean, intuitive design that allows seamless image upload and result viewing. The diagnostic feedback is displayed in a structured format that aligns with clinical expectations, enabling doctors to quickly assess the AI's suggestion and integrate it into their own diagnostic process.

By delivering high diagnostic accuracy (over 97%) and transparent output, the website bridges the gap between artificial intelligence and clinical trust. It does not replace the doctor's judgment but acts as a supplementary tool to reduce diagnostic errors, streamline patient triage, and facilitate early intervention—especially in time-sensitive or resource-constrained settings.

Mobile Application

1. Overview

The mobile application serves as a user-friendly interface for skin cancer detection, enabling users to upload images and receive predictions for melanoma, nevus, and basal cell carcinoma (BCC). The integration of the trained MobileNetV2 model ensures real-time, accurate results, bridging the gap between advanced AI technology and practical healthcare applications.

2. Key Features

1. Image Upload:

Users can upload images of skin lesions directly from their device's gallery for analysis. This feature ensures flexibility, allowing users to choose pre-existing images for evaluation.

2. Real-Time Image Capture:

The app also supports capturing images in real time using the device's camera. This ensures users can quickly and conveniently submit fresh images for analysis without leaving the application.

3. Prediction Results:

The application processes the uploaded or captured image and provides clear, concise predictions. It identifies the likelihood of melanoma, nevus, or basal cell carcinoma (BCC), with confidence scores to aid user understanding.

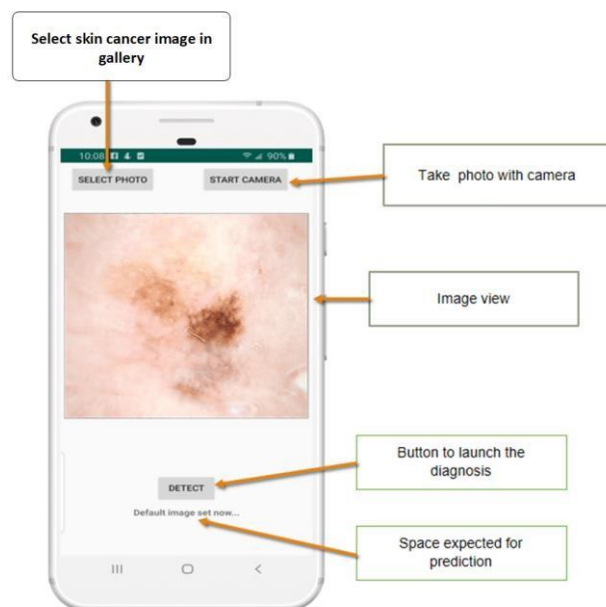


Figure 21 Features that our mobile application offers

4. Ease of Use:

Designed with simplicity in mind, the app features an intuitive interface. Users with minimal technical expertise can easily navigate the app, upload or capture images, and access prediction results

3. Mobile Application in Action

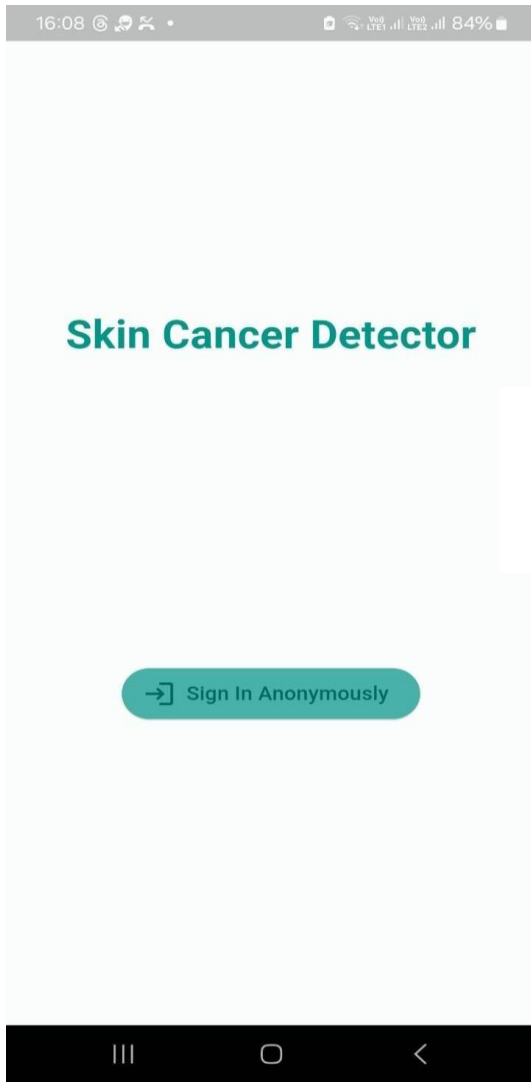


Figure 22 Screen1- Sign in page

Description:

This screen is the first screen the user can see it allows him to sign in to the system anonymously.



Figure 23 Screen 2- Image Selection

Description:

This screen allows users to upload or capture a skin lesion image. Once an image is provided, the "Detect" button becomes active, enabling users to proceed to analysis.

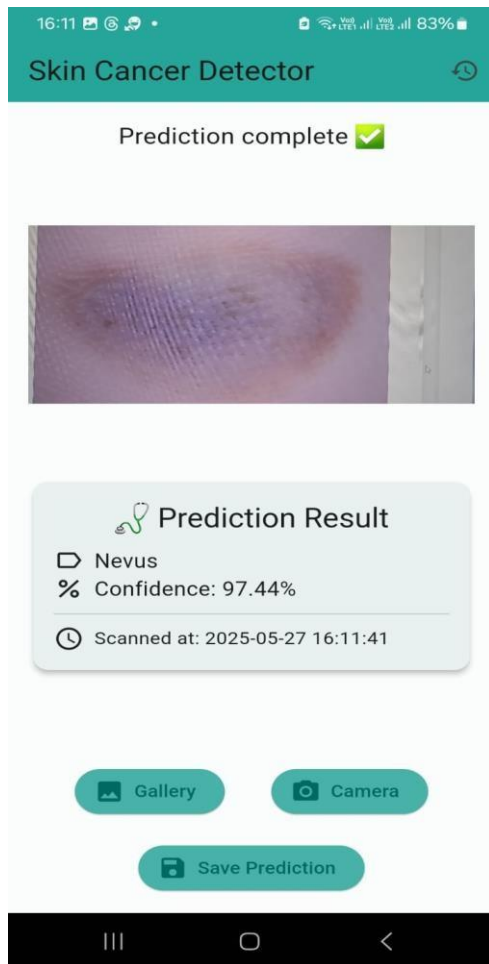


Figure 24 Screen 3 – Results Display

Description:

After analysis, the screen shows the predicted skin lesion type (melanoma, nevus, or BCC), the confidence scores for each class, and the exact date and time the scan was performed.

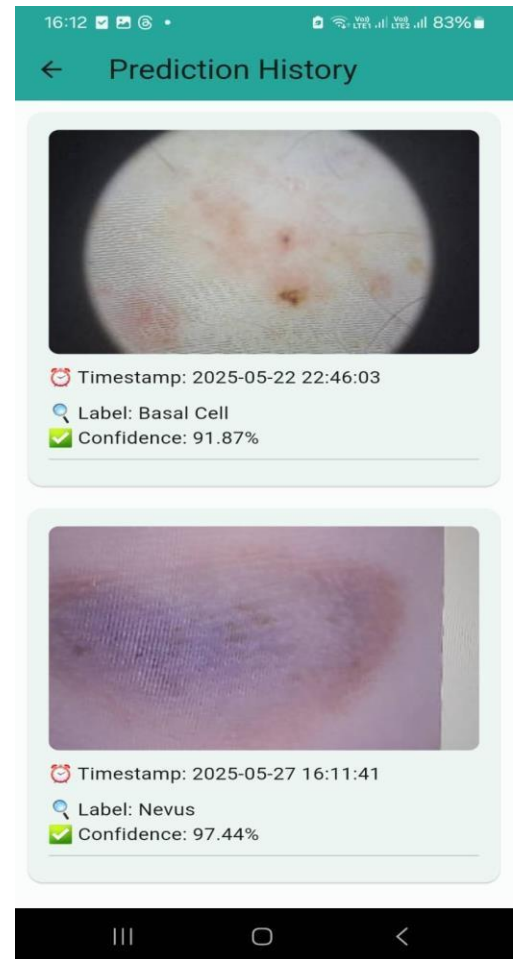


Figure 25 Screen4- Saving the prediction

Description:

After receiving the prediction results, the user can click the **"Save Prediction"** button to store the results securely in Firebase.

Conclusion

This project showcases the practical application of deep learning in the early and accessible detection of skin cancer, specifically targeting **melanoma**, **nevus**, and **basal cell carcinoma (BCC)**. Through rigorous model training, consistent preprocessing, and comprehensive evaluation—including accuracy, precision, recall, F1-score, confusion matrix, and ROC–AUC analysis—the system achieved a high level of performance with **over 97% accuracy**.

By deploying this model across **two platforms**—a **clinic-oriented website** and a **user-friendly mobile application**—the project bridges the gap between advanced AI diagnostics and real-world healthcare needs. The web interface aids medical professionals by providing interpretable outputs, such as heatmaps and detailed confidence scores, while the mobile app empowers everyday users to monitor their skin lesions conveniently.

While the results are promising, the model is not intended to replace clinical judgment. Instead, it serves as a **complementary diagnostic tool**, designed to support early detection, reduce diagnostic delays, and increase healthcare accessibility—especially in resource-limited environments.

This project marks a meaningful step forward in the integration of AI into healthcare, demonstrating how machine learning can enhance both preventive care and clinical decision-making

References

- International Skin Imaging Collaboration (ISIC) 2019 Challenge dataset. Available at: <https://challenge.isic-archive.com/data/>
- Naeem, M., et al. (2025). *A robust deep learning framework for multiclass skin cancer classification using SCDNet*. Scientific Reports, 15, 89230. <https://www.nature.com/articles/s41598-025-89230-7>
- Alahmadi, A., et al. (2024). *Diagnosis of Skin Cancer Using VGG16 and VGG19 Based Transfer Learning*. arXiv preprint arXiv:2404.01160. <https://arxiv.org/html/2404.01160v1>
- Harahap, R., et al. (2024). *Skin cancer classification using EfficientNet architecture*. ResearchGate. https://www.researchgate.net/publication/382776445_Skin_cancer_classification_using_EfficientNet_architecture
- Raj, A., et al. (2025). *Implementation InceptionV3 + SVM Model to Achieve High Accuracy for Skin Cancer Classification*. In: Kumar, S., et al. (eds) Fifth Congress on Intelligent Systems. Lecture Notes in Networks and Systems, vol 1275. Springer, Singapore. https://doi.org/10.1007/978-981-96-2694-6_29
- Zare, R., & Pourkazemi, A. (2021). *DenseNet approach to segmentation and classification of dermoscopic skin lesions images*. arXiv preprint arXiv:2110.04632. <https://arxiv.org/abs/2110.04632>
- Rokad, B., & Nagarajan, S. (2019). *Skin Cancer Recognition using Deep Residual Network*. arXiv preprint arXiv:1905.08610. <https://arxiv.org/abs/1905.08610>
- Mirunalini, P., et al. (2017). *Deep Learning for Skin Lesion Classification*. arXiv preprint arXiv:1703.04364. <https://arxiv.org/abs/1703.04364>